

TOPIC 01

INTRODUCTION TO INFORMATION TECHNOLOGY

Fundamentals of Computing
(FSPK0022)

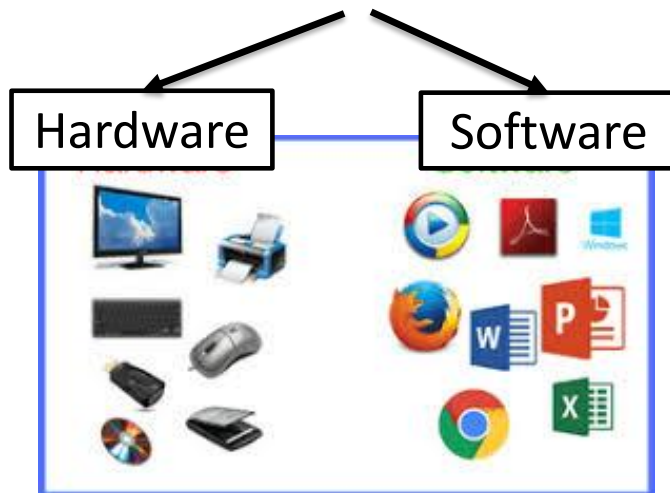
Computer

- Computer is **programmed device** with a **set of instructions** to **perform specific tasks** and **generate results** at a very high speed.
- Computer function:
 - Solve different problems
 - Process data
 - Store & retrieve data
 - Perform calculations faster and efficiently than human.
- The **first computers** were **people who did computations**.
 - Those who compute are computers $\Rightarrow$ just like those who lecture are lecturer.



Harvard Computers

Computer



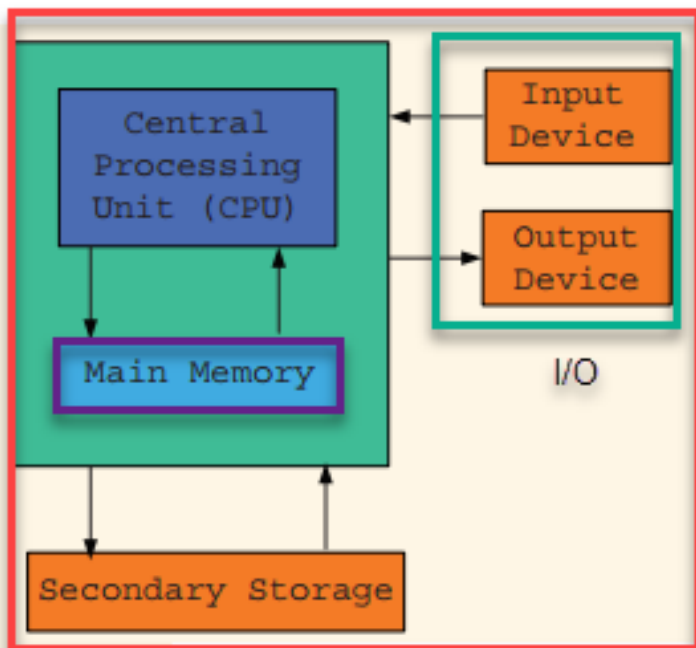
Hardware

- Collection of **physical elements** that constitutes a computer system.
- Refers to physical parts or components of a computer.

Software

- **Set of instructions/** programs instructing a computer to do specific tasks.
- Generic term for organized collections of computer data and instructions.

Hardware Components



address	content
	program & data
.	.
.	.
.	.
1000	54
1001	A
.	.
.	.
.	.
2000	.
2001	.
.	.
.	.
.	.

Main Memory

Hardware components of a computer and main memory

Hardware Components

Central processing unit (CPU)

- The **brain** of a computer
- In general, the more powerful the CPU, the faster the computer.
- Arithmetic and logical operations are carried out inside the CPU.



Main Memory or Random Access Memory (RAM)



- Is an **ordered sequence of cells**, called memory cells and connected directly to the CPU ⇒ each cell has a **unique location** in main memory, called the address of the cell.
- Stores everything as sequences of 0s and 1s. The memory addresses are also expressed as sequences of 0s and 1s.
- All programs must be loaded into main memory before they can be executed. ⇒ Similarly, all data must be brought into main memory before a program can manipulate it.
- When the computer is turned off, everything in main memory will be lost.

Hardware Components

Secondary Storage

- Information stored in main memory must be saved in some other device for permanent storage.
- Because everything in main memory will be lost when the computer is turned off.
- Stores information permanently.
- Example: hard disks, flash drives, and CD-ROMs.



Hardware Components

Input

- Devices that feed data into computers.
- e.g keyboard, mouse, scanner, camera.



Output

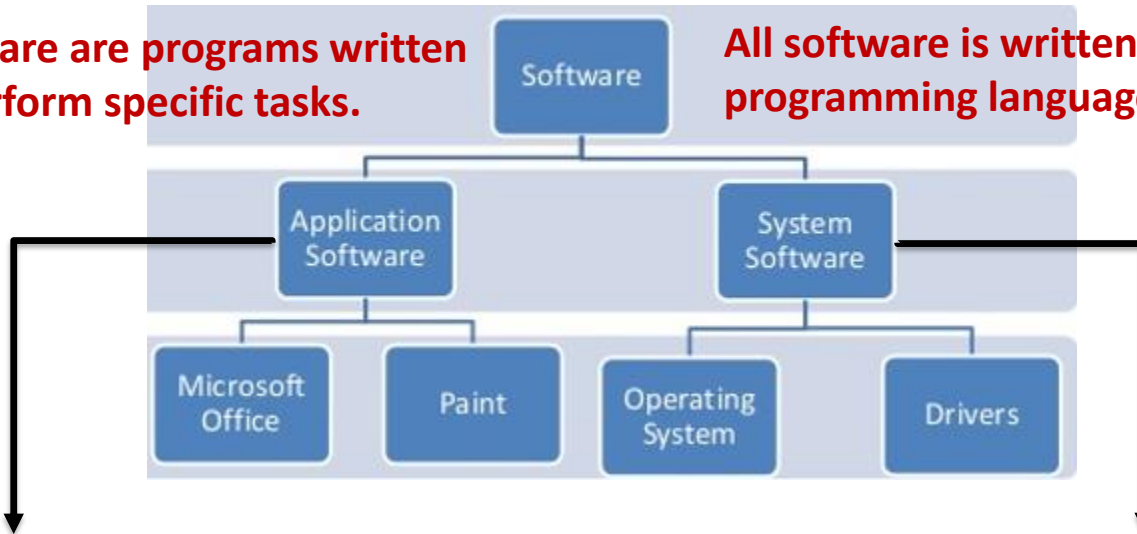
- Devices use to display results
- e.g monitor, printer.



Software

Software are programs written to perform specific tasks.

All software is written in programming languages.

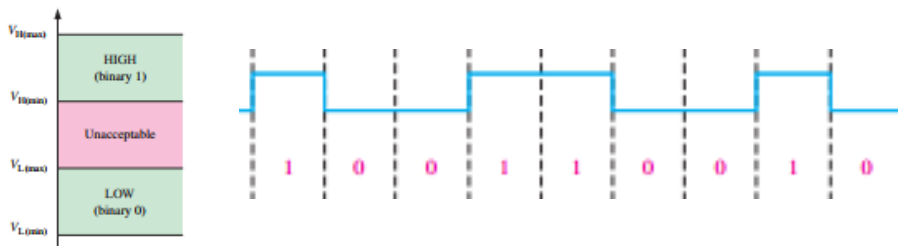


Application (or app for short) is computer software designed to perform a group of coordinated functions, tasks, or activities **for the benefit of the end user**. e.g. Photoshop, Word, Excel, MySQL

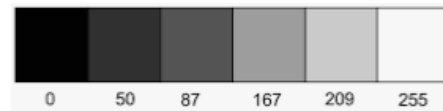
System software responsible for **controlling, integrating, and managing** the individual **hardware components** of a computer system. e.g. operating system, utilities such as disk formatters, file managers, display managers, text editors, compiler, device driver

How Data is Represented on a Computer

- Digital signals represent information with a sequence of 0s and 1s
 - 0 represents a low voltage, and 1 represents a high voltage



- The smallest unit of data in a computer is a bit
- A **bit** is a single **binary digit**, with either 1 (on) or 0 (off) value.
- Eight bits can combine to make a **byte**, which is an 8-digit binary number
- more bits will represent more information
 - single bit – 2 possibility – 0 or 1 (can represent 0 - close, 1 - open, or 0 - false, 1 - true)
 - 8 bits – represent $2^8 = 256$ representation
 - e.g. no of student 0 .. 255
 - brightness: gray scale $\Rightarrow$ 0 - dark , 255 - brightest



How Data is Represented on a Computer

Unit	Symbol	Bits/Bytes
Byte		8 bits
Kilobyte	KB	2^{10} bytes = 1024 bytes
Megabyte	MB	$1024 \text{ KB} = 2^{10} \text{ KB} = 2^{20}$ bytes = 1,048,576 bytes
Gigabyte	GB	$1024 \text{ MB} = 2^{10} \text{ MB} = 2^{30}$ bytes = 1,073,741,824 bytes
Terabyte	TB	$1024 \text{ GB} = 2^{10} \text{ GB} = 2^{40}$ bytes = 1,099,511,627,776 bytes
Petabyte	PB	$1024 \text{ TB} = 2^{10} \text{ TB} = 2^{50}$ bytes = 1,125,899,906,842,624 bytes
Exabyte	EB	$1024 \text{ PB} = 2^{10} \text{ PB} = 2^{60}$ bytes = 1,152,921,504,606,846,976 bytes
Zettabyte	ZB	$1024 \text{ EB} = 2^{10} \text{ EB} = 2^{70}$ bytes = 1,180,591,620,717,411,303,424 bytes

Note:

$\text{KB} = 2^{10} = 1024 \text{ byte} = 1 \text{ kilobyte}$

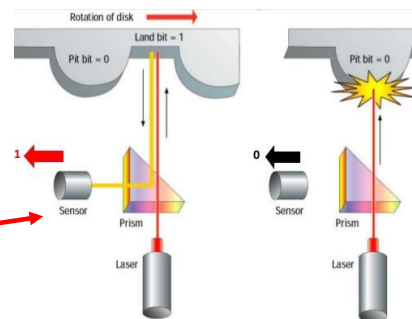
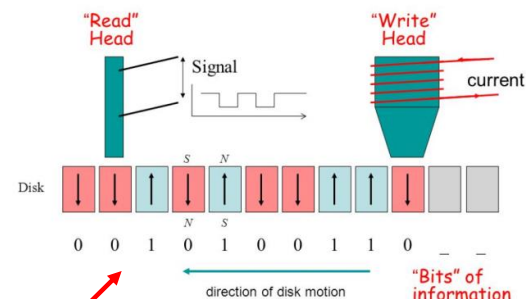
$\text{KB} = 10^3 = 1000 \text{ byte} = 1 \text{ kilobyte}$

Concept of Data Representation in Computer

- Computer is a **numerical machine**. It can only process in binary and in a form of numbers.
- Any type of data $\Rightarrow$ numbers, letters, special symbols, sound or pictures must be **converted to binary** form.
- Data representation
 - Electronic circuit:** using voltage – High (1) and Low (0).
 - Magnetic media** e.g. hard disk - The presence of a magnetic field in one direction on magnetic media is interpreted as 1; while the field in the opposite direction is interpreted as 0
 - Optical devices** e.g. DVD - the presence of light is interpreted as 1, while its absence is interpreted as 0

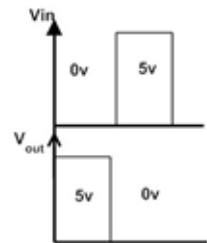
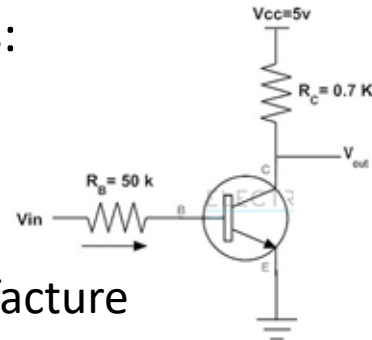
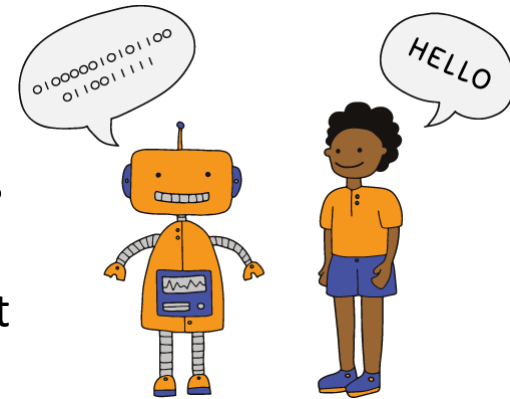


A computer hard drive stores your data **magnetically**



Why Computer Use Binary?

- Natural language (human language) – too difficult
- Computer circuit behave like a switch (ON or OFF)
 - Binary in nature. Has only two states (two possibility): 1 or 0.
- Decimal – difficult because has 10 possibilities (0..9)
 - Earlier computer development uses decimal but finally all computers use binary
- Binary digital system has an advantages:
 - Digital computer more reliable
 - Smaller than analog devices
 - Uses less energy
 - Digital Integrated circuit easier to manufacture



Number System



Number system is a set of symbols used to represent values derived from common base or radix

Group it as

Binary
Octal
Hexadecimal



For Computer: 2 major category

Base 10 (**Decimal**) – 0 .. 9

Octal and Hexadecimal are derived from base 2

- Binary (base 2) , digit: 0,1 – written as X_2 or X_{bin}
- Octal - base 8, digit: 0 ..7 - written as X_8 or X_{oct}
- Hexadecimal – base 16, digit 0..9, A..F - written as X_{16} or X_{hex}



Octal and Hexadecimal use to reduce the streams of binary digits into manageable form

For user - Easy to use than binary, less prone to error

- e.g $C9BF3E_{16}$ easier than
 110010011011111
 100111110₂

Easier to write in program



In programming language to specify different number system, number written as



Binary literals:	0B110101111	0b100100011U	0b1010L	0B110010101101	0b11111111
Octal literals:	0657	0443U	012L	06255	0377
Hexadecimal literals:	0x1AF	0x123U	0xAL	0xcd	0xFF

C++ Primitive Data Type: Range of Value

Type	Typical Bit Width	Typical Range
char	1byte	-127 to 127 or 0 to 255
unsigned char	1byte	0 to 255
signed char	1byte	-127 to 127
int	4bytes	-2147483648 to 2147483647
unsigned int	4bytes	0 to 4294967295
signed int	4bytes	-2147483648 to 2147483647
short int	2bytes	-32768 to 32767
unsigned short int	Range	0 to 65,535
signed short int	Range	-32768 to 32767
long int	4bytes	-2,147,483,648 to 2,147,483,647
signed long int	4bytes	same as long int
unsigned long int	4bytes	0 to 4,294,967,295
float	4bytes	+/- 3.4e +/- 38 (~7 digits)
double	8bytes	+/- 1.7e +/- 308 (~15 digits)
long double	8bytes	+/- 1.7e +/- 308 (~15 digits)
wchar_t	2 or 4 bytes	1 wide character

- Integer
- Character
- Boolean
- Floating Point
- Double Floating Point
- Valueless or Void
- Wide Character

The size of variables might be different from those shown in the above table, depending on the compiler and the computer you are using.

Integer Representation

- Unsigned: zero, positive value only
 - For n bit representation: range of value $0.. 2^n-1$
 e.g. in C++ , unsigned short int represented as 16 bit, range 0 .. 65535
 - Each integer value converted directly to binary equivalent
 e.g. for unsigned short int, 29 will be represented as

0000 0000 0001 1101₂

Unsigned integer

Integer

- Signed: negative, zero, positive value
 - For n bit representation: range of value $- 2^{n-1} ..+2^{n-1} -1$
 e.g. in C++ , short int is equivalent to signed short int, represented as 16 bits, range -32768 ..+32767
 - Each integer will be converted to 2's complement form

Signed integer

Sign Integer

Positive number: sign = 0, integer (magnitude) in binary

Negative number: sign = 1, integer (magnitude) in 2's complement

2's Complement

- Several method to get the 2's complement

- **Method 1:**

1. Find the 1's complement (change 0 to 1 and 1 to 0) of the number and then add 1
2. The 2's complement of $101100 = 010011 + 1 = 010100$

- **Method 2:**

1. Scan from right to left: leave all trailing zeroes unchanged
2. keep the first 1; reverse the remaining 0s and 1s to the left

- Example: Number stored in a computer memory as

1100 0110 1001 1110₂, determine its value.

2's Complement

1100 0110 1001 1110

MSB (most significant bit) $\Rightarrow$ sign bit
 0 – the number is positive
 1 – the number is negative

e.g.

0100 0110 1001 1110₂

MSB = 0, positive number

magnitude: convert binary to decimal

0100 0110 1001 1110 = + 18078

If sign bit negative,
 magnitude in 2's complement take the 2's
 complement of the number to get it actual
 value

invert all bit - 0011 1001 0110 0001

add 1 1

0011 1001 0110 0010

= - 14690

How Representation Stored in Memory

- Different processor has different way of storing a number bigger than byte size in computer memory
 - Intel use Little Endian, Atmel AVR32 use big endian, ARM6 onward – can operate on both big and little endian.



BIG ENDIAN - The way people always broke their eggs in the Lilliput land



LITTLE ENDIAN - The way the king then ordered the people to break their eggs

0x12	0x34	0x56	0x78
------	------	------	------

Big Endian

0x78	0x56	0x34	0x12
------	------	------	------

Little Endian

Suppose x is a 4-byte integer:

```
int x = 0x01234567;
```

and the address of x is 0x100.

Note: X in binary: 0000 0001 0010 0011 0100 0101 0110 0111

Big endian byte order:

Address	0x100	0x101	0x102	0x103		
Data	...	0000 0001	0010 0011	0100 0101	0110 0111	...

01

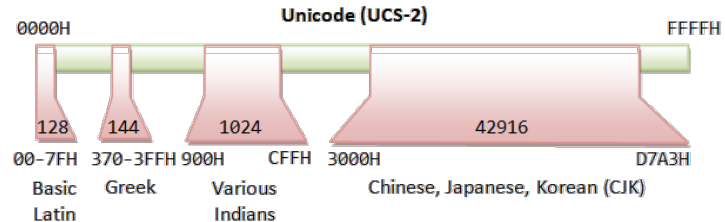
Little endian byte order:

Address	0x100	0x101	0x102	0x103	
Data	...	0110 0111	0100 0101	0010 0011	0000 0001 ...

67

Symbolic Representation using Coding

- In computing, single character, number or symbol is represented by a group of bits
 - commonly used encoding scheme on personal computers is the seven-bit American Standard Code for Information Interchange (**ASCII**)
 - character A is encoded as the binary number 1000001
- Other encoding schemes:
 - EBCDIC** (used by IBM)
 - ❖ Consists of 256 characters;
 - Unicode**, which is a more recent development
 - ❖ Unicode consists of 65,536 characters (UCS-2)
 - ❖ Created to represent a variety of characters and is continuously expanding.
 - ❖ It consists of characters from languages other than English



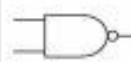
ASCII Table

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Computer Basic Building Blocks

- Logic gates are the most basic building block for any digital system
- Need to know the logic characteristic because logic is part of a programming language
 - Example: `if (x<2) && (y>10) then`

****Note:** `&&` is AND operator

Name	NOT	AND	NAND	OR	NOR	XOR	XNOR																																																																																																
Alg. Expr.	$\overline{A}$	AB	$\overline{AB}$	$A+B$	$\overline{A+B}$	$A\oplus B$	$\overline{A\oplus B}$																																																																																																
Symbol																																																																																																							
Truth Table	<table><tr><th>A</th><th>X</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	X	0	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	X	0	0	0	0	1	0	1	0	0	1	1	1	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	X	0	0	1	0	1	1	1	0	1	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	X	0	0	0	0	1	1	1	0	1	1	1	1	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	X	0	0	1	0	1	0	1	0	0	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	B	A	X	0	0	0	0	1	1	1	0	1	1	1	0	<table><tr><th>B</th><th>A</th><th>X</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	B	A	X	0	0	1	0	1	0	1	0	0	1	1	1
A	X																																																																																																						
0	1																																																																																																						
1	0																																																																																																						
B	A	X																																																																																																					
0	0	0																																																																																																					
0	1	0																																																																																																					
1	0	0																																																																																																					
1	1	1																																																																																																					
B	A	X																																																																																																					
0	0	1																																																																																																					
0	1	1																																																																																																					
1	0	1																																																																																																					
1	1	0																																																																																																					
B	A	X																																																																																																					
0	0	0																																																																																																					
0	1	1																																																																																																					
1	0	1																																																																																																					
1	1	1																																																																																																					
B	A	X																																																																																																					
0	0	1																																																																																																					
0	1	0																																																																																																					
1	0	0																																																																																																					
1	1	0																																																																																																					
B	A	X																																																																																																					
0	0	0																																																																																																					
0	1	1																																																																																																					
1	0	1																																																																																																					
1	1	0																																																																																																					
B	A	X																																																																																																					
0	0	1																																																																																																					
0	1	0																																																																																																					
1	0	0																																																																																																					
1	1	1																																																																																																					

Computer Basic Building Blocks

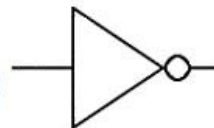
3 most basic gates

- **NOT** - output is the complement of input
- **AND** - 0 (LOW, FALSE) at the input the output zero
- **OR** - 1 (HIGH, TRUE) at the input the output 1

The Logical Operators

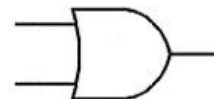
Operator	Symbol	Example
AND	&&	<code>grade >= 70 && grade < 80</code>
OR		<code>grade > 100 grade < 1</code>
NOT	!	<code>!grade >= 70</code>

Inverter
(or NOT gate)



in	out
1	0
0	1

OR gate



in 1	in 2	out
0	0	0
1	0	1
0	1	1
1	1	1

AND gate



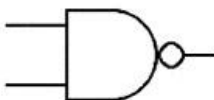
in 1	in 2	out
0	0	0
1	0	0
0	1	0
1	1	1

NOR gate



in 1	in 2	out
0	0	1
1	0	0
0	1	0
1	1	0

NAND gate



in 1	in 2	out
0	0	1
1	0	1
0	1	1
1	1	0

**A computer once beat me at
chess, but it was no match for me
at kick boxing.**

Emo Philips

